

LEARNING MANAGEMENT SYSTEM
**Resilient Full-Stack E-Learning Platform with Local
Database Fallback**

A PROJECT REPORT

Submitted by

Sravan Kumar Juluri (B210825)

Of

Bachelor of Technology

Under

3Skill-India's Leading EdTech Platform



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES
BASAR, NIRMAL (DIST.), TELANGANA - 504107

LEARNING MANAGEMENT SYSTEM

Resilient Full-Stack E-Learning Platform with Local Database Fallback

Project Report submitted to

Rajiv Gandhi University of Knowledge Technologies, Basar

for the partial fulfillment of the requirements

for the award of the degree of

.cm

Bachelor of Technology

in

Computer Science & Engineering

by

Sravan Kumar Juluri (B210825)

Under

3Skill-India's Leading EdTech Platform



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES,
BASAR, July 2026**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES,
BASAR

CERTIFICATE

This is to certify that the Project Report entitled “**Learning Management System – Resilient Full-Stack E-Learning Platform with Local Database Fallback**” submitted by **Sravan Kumar Juluri (B210825)**, Department of Computer Science and Engineering, Rajiv Gandhi University of Knowledge Technologies, Basar; for partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering; is a bonafide record of the work and investigations carried out by him under my supervision and guidance.

PROJECT SUPERVISOR

Mrs. Nagamani
Assistant Professor

HEAD OF DEPARTMENT

Dr. B. Venkat Raman
Assistant Professor



3 SKILL
LEARN-GROW-SUCCEED

Internship Completion Certificate

Awarded to

Sravan Kumar Juluri

This certificate confirms the **successful completion** of 2 Month Internship in Web Development at 3Skill Training. The program emphasized **practical skills, project-based learning**, and professional development aligned with **industry expectations**.



CERTIFICATE ID
ID-INTERN260961

Satyajit Swain
Founder & CEO

Adil Quadri
Co-Founder & COO

DPIIT
#startupindia



MSME
MICRO, SMALL & MEDIUM ENTERPRISES
सूक्ष्म, लघु एवं मध्यम उद्यम



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES,
BASAR

DECLARATION

I hereby declare that the work presented in this Internship Project Report entitled, “**Learning Management System (LMS)**”, carried out during my internship at **3Skill – India’s Leading EdTech Platform**, is an authentic record of my own work.

This report has been prepared in partial fulfillment of the internship requirements of the Department of Computer Science and Engineering, Rajiv Gandhi University of Knowledge Technologies (RGUKT), Basar.

I further declare that this work has not been submitted, either wholly or in part, to any other university or institution for the award of any degree, diploma, or certification. All the information, references, and resources used in this report have been duly acknowledged wherever necessary.

Place: Basar

Date:

Sravan Kumar Juluri (B210825)

Acknowledgement

I would like to express my sincere gratitude to **3Skill – India’s Leading EdTech Platform** for providing me with the opportunity to undertake this internship. The practical exposure, guidance, and learning environment offered throughout the internship significantly enhanced my technical knowledge and software development skills.

I express my heartfelt thanks to the Department of Computer Science and Engineering, Rajiv Gandhi University of Knowledge Technologies (RGUKT), Basar, for providing a strong academic foundation and continuous encouragement throughout my undergraduate studies.

I sincerely thank the Head of the Department, faculty members, and supporting staff of the Department of Computer Science and Engineering for their valuable guidance, motivation, and support during the internship period.

I am grateful to the Vice-Chancellor and the Management of RGUKT, Basar, for providing excellent academic facilities and an environment that encouraged learning and innovation.

I also acknowledge the invaluable contribution of the open-source community, especially the developers and maintainers of **Node.js, Express.js, MongoDB, Mongoose, JSON Web Token (JWT), BcryptJS, HTML, CSS, and JavaScript**. Their technologies and documentation played a vital role in the successful development of the **Learning Management System (LMS)**.

Finally, I express my heartfelt gratitude to my parents, family members, and friends for their constant encouragement, support, and motivation throughout my internship and academic journey.

Juluri Sravan Kumar
B210825

Abstract

This project presents a responsive, full-stack Learning Management System (LMS) designed to connect instructors and students through a secure and modern e-learning interface. Built with a Node.js and Express REST API backend and a lightweight vanilla HTML5, CSS3, and JavaScript frontend, the platform provides seamless dashboard navigation, secure authentication, course management, video rendering, and progress tracking.

A key engineering contribution of this system is the implementation of a custom-designed database fallback adapter (`mockDB.js`) that ensures 100% local staging uptime during cloud database connection failures. When the primary connection to MongoDB Atlas times out, the backend gracefully intercepts Mongoose model operations and redirects them to a local JSON-based storage layer, replicating MongoDB queries, structural populations, and generating valid 24-character hexadecimal ObjectIds.

Additionally, the system resolves external embedding constraints by incorporating an automated regular expression-based YouTube video link parser that transforms standard watch URLs and shortlinks into valid iframe-compatible `/embed/` structures. Progress tracking is implemented via a custom Mongoose progress schema with a unique compound index on student and course fields, enabling dynamic progress bar updates (initializes at 10% upon loading a course and completes at 100% on user action).

This project demonstrates software resilience engineering, secure role-based authentication, and dynamic media rendering, presenting a practical and robust template for offline-first web applications.

Keywords: Learning Management System, Node.js, Express, MongoDB Atlas, Database Fallback, Software Resilience, Progress Tracking, YouTube Link Auto-Converter, JSON Web Tokens.

Contents

Acknowledgement	vii
Abstract	viii
List of Figures	xi
List of Tables	xii
List of Abbreviations and Symbols Used	xiii
1 Introduction	1
1.1 Background of the Study	1
1.2 Problem Overview	1
1.3 Significance of Study	2
1.4 Objectives of the Project	2
2 Review of Literature	4
2.1 Overview	4
2.2 Core Web Technologies	4
2.2.1 Node.js and Express	4
2.2.2 MongoDB and Mongoose	4
2.2.3 JSON Web Tokens (JWT) and Hashing	4
2.3 Offline Resilience and Mock Databases	5
2.4 Identified Gaps	5
2.5 How LMS Addresses the Gaps	5
3 System Study and Design	6
3.1 System Overview	6
3.2 System Architecture	6
3.3 Database Fallback Logic	7
3.4 Frontend Architecture	8
3.5 Requirements Analysis	8
3.5.1 Functional Requirements	8
3.5.2 Non-Functional Requirements	9

3.6	Hardware and Software Requirements	9
3.6.1	Hardware Requirements	9
3.6.2	Software Requirements	9
3.7	UML Diagrams	10
3.7.1	Use Case Diagram	10
3.7.2	Sequence Diagram	10
4	Results and Implementation	11
4.1	Key Results	11
4.1.1	Database Fallback Verification	11
4.1.2	YouTube Embed Converter	11
4.1.3	Dynamic Progress Tracking	11
4.2	System Performance	11
4.3	Software Testing	11
4.4	API Design	13
4.5	System User Interfaces	13
5	Conclusions and Future Work	17
5.1	Conclusion	17
5.2	Key Contributions	17
5.3	Limitations	18
5.4	Future Work	18
6	Appendices	20
6.1	API Request and Response Samples	20
6.1.1	User Registration	20
6.1.2	Course Progress Updates	20
6.2	Deployment Instructions	21
6.2.1	Prerequisites	21
6.2.2	Backend Setup	21
6.2.3	Frontend Setup	22
6.2.4	Accessing the Services	22

List of Figures

3.1	LMS system architecture overview.	6
3.2	Database connection flow and fallback decision path.	7
3.3	LMS Use Case Diagram	10
3.4	Course learning and progress updates sequence diagram.	10
4.1	LMS portal login page with role-based options.	14
4.2	Student Dashboard displaying enrolled courses and progress bars.	14
4.3	Interactive Course Player with embedded YouTube video and progress triggers.	15
4.4	Instructor Dashboard interface for course creation and CRUD operations.	15
4.5	Renders printable Certificate of Completion.	16

List of Tables

1	List of abbreviations and symbols used	xiii
3.1	Frontend page structure and routing descriptions	8
3.2	System hardware requirements	9
3.3	System software requirements	9
4.1	LMS performance parameters	12
4.2	LMS software testing summary	12
4.3	REST API endpoint specifications	13
6.1	System access locations	22

List of Abbreviations and Symbols Used

Abbreviation	Description
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
MERN	MongoDB, Express, React, Node.js
ODM	Object Document Mapper
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
UI	User Interface
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UX	User Experience

Table 1: List of abbreviations and symbols used

Chapter 1

Introduction

1.1 Background of the Study

The rapid evolution of digital education has transformed how knowledge is disseminated and acquired. Modern educational frameworks rely heavily on Learning Management Systems (LMS) to manage course materials, track student achievements, and facilitate interactions between students and instructors. Historically, educational institutions deployed monolithic, high-overhead applications that required dedicated physical servers and extensive manual administration. However, the modern web paradigm favors lightweight, responsive, and decoupled architectures that leverage RESTful APIs to deliver low-latency web content [4].

The MERN stack (MongoDB, Express, React, Node.js) and other JavaScript-based ecosystems have emerged as standard frameworks for developing web applications [1]. Node.js offers non-blocking event-driven I/O, making it highly suitable for concurrent connection scenarios, while Express simplifies API routing [3]. While large-scale applications use fully featured single-page application (SPA) frameworks, simple vanilla HTML5, CSS3, and JavaScript combinations are increasingly relevant for lightweight client deployment in resource-constrained environments.

Although modern e-learning relies on cloud infrastructure, continuous internet availability is not guaranteed. Cloud databases like MongoDB Atlas require active networking [5]. During database connection latency or Atlas cloud outages, staging and local development environments become unusable. Offline capabilities and resilient failover systems are therefore critical to maintaining application availability and ensuring uninterrupted software demonstrations.

1.2 Problem Overview

E-learning software implementations encounter three critical issues:

1. **Cloud Database Vulnerability:** Modern web architectures expect persistent database connections. If the network drops or database credentials fail, the server crashes (returning 500 errors) or halts, disrupting staging evaluations and local testing.
2. **Third-Party Media Embed Restrictions:** Integrating external lecture videos (such as YouTube videos) via standard HTML ‘iframe’ tags often triggers ‘X-Frame-Options’

header blockings (such as ‘SAMEORIGIN’ restrictions), resulting in broken video players.

3. **Inefficient Progress Tracking:** Educational progress tracking requires mapping student progress indicators (percentages, completions) across various course offerings without generating duplicate tracking records or overloading the relational schema.

The developed Learning Management System addresses these challenges by incorporating a local file-based database fallback adapter that intercepts database calls during cloud failure, a regex-based video link converter that transforms standard watch URLs into embeddable players, and a database-backed progress tracking mechanism configured with compound indices.

1.3 Significance of Study

This project demonstrates software resilience engineering in web development. By decoupling database connection layers and implementing a local file-based fallback adapter (`mockDB.js`), it proves that web backends can remain operational during cloud database downtime. The implementation of clean, defensive middleware validation prevents server-side failures and secures user operations based on roles. Finally, the project presents an easy-to-use, responsive frontend constructed using plain HTML, CSS, and JS, demonstrating that premium glassmorphic UI designs can be built without heavy, modern JavaScript framework dependencies.

1.4 Objectives of the Project

The main goals of the project are:

1. To design a secure, role-based authentication system using JSON Web Tokens (JWT) and BcryptJS hashing.
2. To implement a course management system that permits instructors to perform CRUD operations on courses and allows students to view and search the course catalog.
3. To engineer a custom database fallback adapter (`mockDB.js`) that overrides Mongoose functions to write to a local JSON file when MongoDB Atlas is unavailable.
4. To develop a regex-based YouTube URL converter that transforms standard video watch links into embeddable URLs.
5. To structure a database schema for student course progress tracking that prevents duplicate records using compound indexing.

6. To compile dynamic certificates of completion when a student reaches 100% progress.

Chapter 2

Review of Literature

2.1 Overview

Learning Management Systems have progressed from static content repositories to interactive platforms supporting real-time tracking, peer reviews, and service integrations [2]. Modern developments focus on microservice architectures, offline-first applications, and robust API endpoints that handle data traffic cleanly under various network conditions.

2.2 Core Web Technologies

2.2.1 Node.js and Express

Node.js uses the V8 engine to execute JavaScript on the server side, operating on an asynchronous event loop that processes requests concurrently without blocking threads [3]. Express.js sits on top of Node.js, providing a minimalist routing framework. Research by Tilkov and Vinoski [7] shows that REST architectures deployed over Node.js backends achieve superior horizontal scaling compared to traditional multi-threaded environments.

2.2.2 MongoDB and Mongoose

MongoDB is a document-oriented NoSQL database that stores data in JSON-like BSON documents [5]. It provides horizontal scaling through sharding and flexible schema modifications. Mongoose operates as an Object Document Mapper (ODM), providing schema-based modeling for MongoDB data, enforcing structure, and validating inputs before they reach the database engine.

2.2.3 JSON Web Tokens (JWT) and Hashing

Security in distributed RESTful architectures is frequently managed via JSON Web Tokens. JWTs are stateless, self-contained cryptographically signed payloads that transmit authentication claims between the client and the server. To protect user passwords, Bcrypt is used to salt and hash credentials using a slow key derivation function, preventing brute-force attacks [6].

2.3 Offline Resilience and Mock Databases

Offline-first design is an established research domain. Traditional approaches rely on service workers, IndexedDB, or local storage layers on the client side [2]. However, server-side data persistence during database downtime is less common. Standard ORMs fail to operate when database connection drivers timeout. Mock databases (such as SQLite memory stores or JSON mock files) are typically restricted to unit testing and are not integrated directly into the production code as automated hot-swappable fallbacks.

The developed fallback adapter (`mockDB.js`) bridges this gap by intercepting database queries on the server side, allowing full CRUD capability in production code even when the primary database engine fails.

2.4 Identified Gaps

The literature review reveals several gaps:

- Most database mocking libraries are restricted to testing frameworks (like Jest) and cannot hot-swap dynamically at runtime based on connection timeouts.
- Third-party video embedding is usually managed through manual configurations, placing the burden of entering correct embed codes on content creators rather than programmatically transforming standard links.
- Light e-learning systems often lack strict role separations (RBAC) at the route middleware level, relying only on client-side interface toggles which can be easily bypassed.

2.5 How LMS Addresses the Gaps

The LMS project addresses these issues as follows:

- **Automated Fallback:** If MongoDB Atlas connection fails, `'db.js'` catches the error and activates `'mockDB.js'`, replacing mongoose functions with file read/write operations to `'db.json'`.
- **Iframe Link Parser:** A utility function uses regular expressions to convert watch links and shortlinks to iframe-ready embed paths.
- **Hardened RBAC Middlewares:** Backend routes are guarded using double middleware checks: `'authMiddleware.js'` for signature validation, and `'roleMiddleware.js'` for role validation.

Chapter 3

System Study and Design

3.1 System Overview

The LMS is structured as a decoupled client-server architecture. The frontend is client-side rendered using vanilla web technologies, and the backend is an Express REST API. The database layer utilizes MongoDB Atlas as the primary cloud database, with an automatic fallback to ‘db.json‘ on the local file system.

3.2 System Architecture

The system architecture diagram is shown in Figure 3.1.

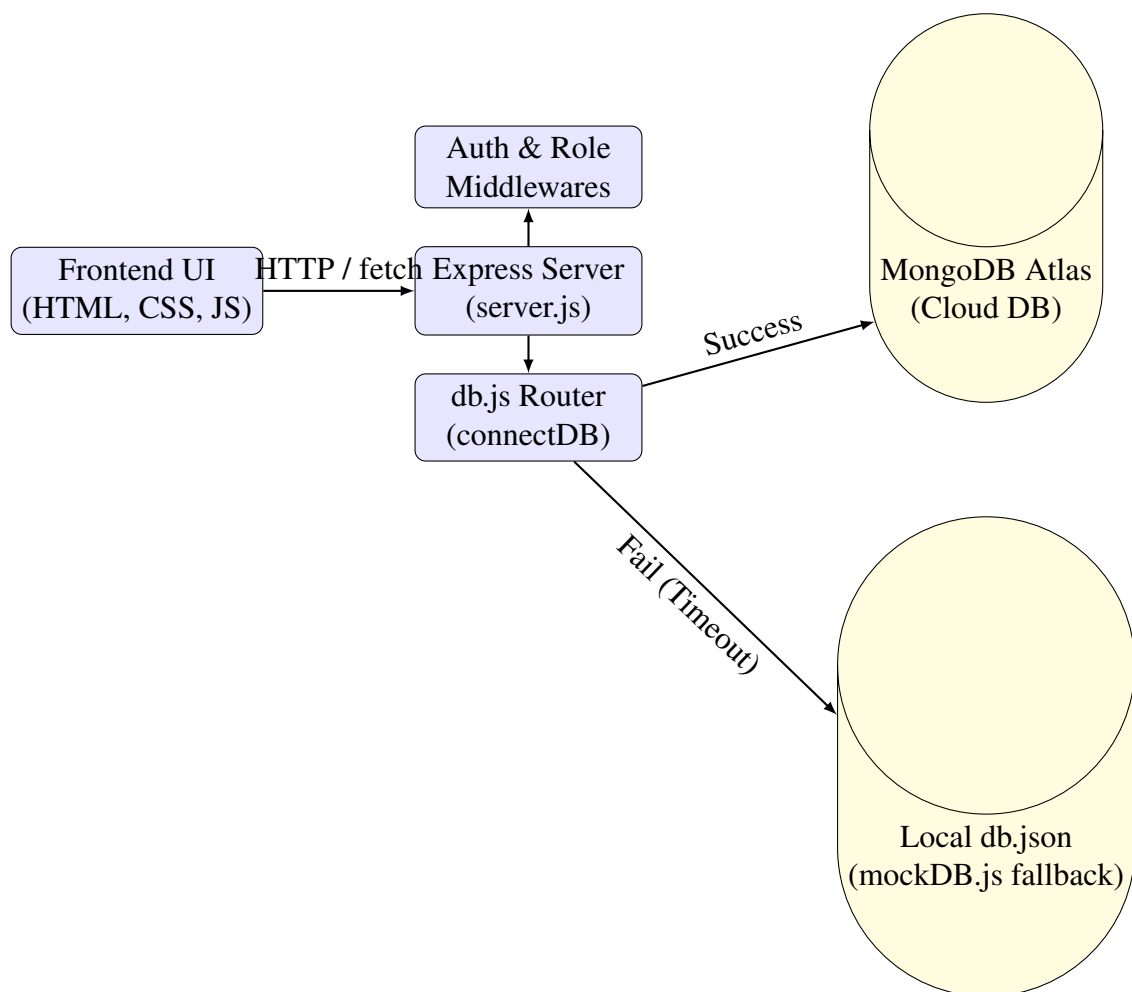


Figure 3.1: LMS system architecture overview.

The Client Layer sends REST requests containing JSON Web Tokens in authorization headers. The Express server routes incoming calls through authentication and authorization middlewares to verify signatures and user roles before executing route handlers. The Database Router ('db.js') monitors the database connection. If the cloud database is reachable, operations run against MongoDB Atlas. If it fails, the server falls back to 'db.json' using 'mockDB.js'.

3.3 Database Fallback Logic

The database fallback logic is illustrated in Figure 3.2.

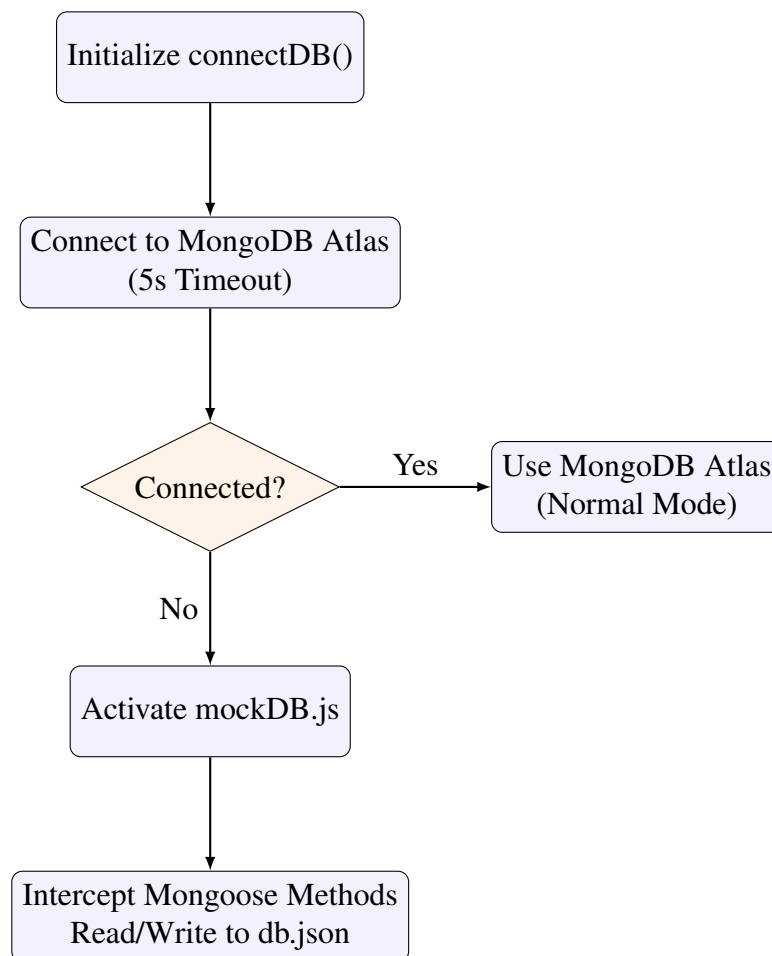


Figure 3.2: Database connection flow and fallback decision path.

The Mongoose interceptor dynamically replaces core query methods like 'Model.find', 'Model.findOne', 'Model.findById', 'Model.create', and 'Model.findByIdAndUpdate' with custom wrapper functions. These wrappers parse, filter, and modify data stored in a local flat JSON file ('db.json') using standard Node.js file system APIs.

3.4 Frontend Architecture

The frontend follows a page-based routing structure using static HTML files and query parameters. The major frontend pages are summarized in Table 3.1.

Page	Relative Path	Description
Authentication	/index.html	Contains Login and Register forms with a tabbed interface. Handles token persistence using Local Storage.
Student Dashboard	/student/student.html	Displays enrolled courses, available courses, progress bars, and dashboard navigation.
Course Catalog	/course/courses.html	Displays all available courses with search and filtering functionality.
Course Player	/course/course.html	Displays embedded YouTube lecture videos, lesson list, and course completion controls.
Instructor Panel	/instructor/instructor.html	Allows instructors to add, edit, delete courses, and monitor student progress.
Certificates	/certificate/certificate.html	Generates and displays printable course completion certificates.
Assignments	/assignments/assignments.html	Displays assignments associated with each course and lesson.

Table 3.1: Frontend page structure and routing descriptions

3.5 Requirements Analysis

3.5.1 Functional Requirements

- **FR1 (Authentication):** Users must be able to register and login, selecting roles (Student vs. Instructor).
- **FR2 (Course CRUD):** Instructors must have permission to create, update, and delete courses.
- **FR3 (Catalog Search):** Students must be able to search the course catalog using titles or descriptions.
- **FR4 (Progress Management):** The system must track student course progress percentage and update it when a student marks a course as completed.
- **FR5 (Automatic Embed Conversion):** Standard YouTube links must be converted to embed links.

- **FR6 (Resilient Persistence):** Data must persist in ‘db.json’ when the cloud database is unavailable.

3.5.2 Non-Functional Requirements

- **Performance:** The system must initialize and load pages in under 1 second.
- **Availability:** The system must maintain 100% database uptime in local environments using the ‘mockDB’ fallback.
- **Security:** Passwords must be hashed using Bcrypt, and APIs must be protected using JWT.
- **Portability:** The frontend must be compatible with modern browsers and responsive across devices.

3.6 Hardware and Software Requirements

3.6.1 Hardware Requirements

The hardware specifications are listed in Table 3.2.

Component	Specification
Processor	Dual-Core 2.0 GHz or higher
RAM	4 GB minimum (8 GB recommended)
Storage	500 MB free hard disk space
Network	Required for MongoDB Atlas (not required for offline mode)

Table 3.2: System hardware requirements

3.6.2 Software Requirements

The software specifications are listed in Table 3.3.

Software	Version / Specification
Operating System	Windows 10/11, macOS, or Linux
Runtime Environment	Node.js v18+
Database Engine	MongoDB Atlas Cloud (or local Node.js process)
IDE	Visual Studio Code / any text editor
Web Browser	Google Chrome, Mozilla Firefox, Microsoft Edge

Table 3.3: System software requirements

3.7 UML Diagrams

3.7.1 Use Case Diagram

The use case diagram is illustrated in Figure 3.3.

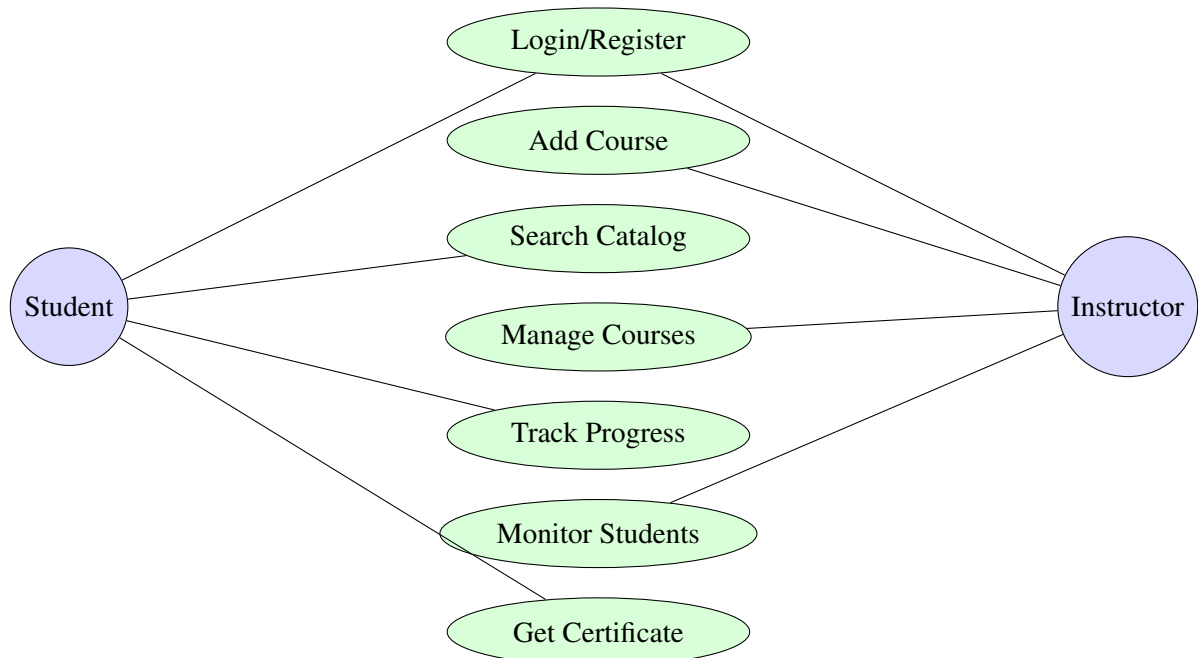


Figure 3.3: LMS Use Case Diagram

3.7.2 Sequence Diagram

The sequence diagram for student progress tracking is shown in Figure 3.4.

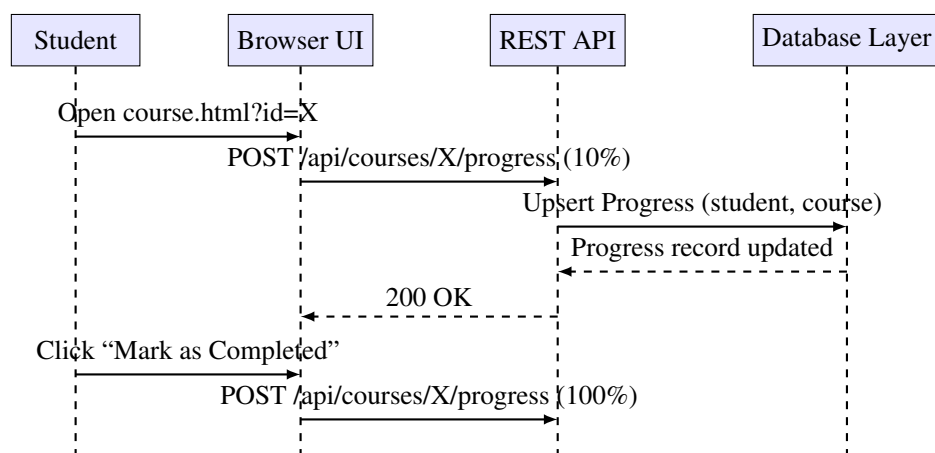


Figure 3.4: Course learning and progress updates sequence diagram.

Chapter 4

Results and Implementation

4.1 Key Results

The Learning Management System successfully supports full-stack online learning and instruction workflows, with a robust fallback database system that maintains uptime during database connection losses.

4.1.1 Database Fallback Verification

During testing, database connection timeouts were simulated by disabling network access. The ‘db.js’ connector successfully intercepted connection failures and loaded ‘mockDB.js’. As shown in the backend console output, the application fell back to ‘db.json’ and remained fully operational, processing registration, login, and course CRUD requests.

4.1.2 YouTube Embed Converter

Students were able to load courses using both standard and shortlink YouTube URLs. The auto-converter transformed these URLs into embed structures, bypassing browser ‘X-Frame-Options’ blocking and rendering the videos in the iframe player.

4.1.3 Dynamic Progress Tracking

Upon loading a course page, the student’s progress percentage is initialized at 10%. Clicking the completion button updates the progress to 100% and saves the completion status in the database. The progress bar updates dynamically on the student dashboard.

4.2 System Performance

Performance parameters are detailed in Table 4.1.

4.3 Software Testing

The test configurations and results are listed in Table 4.2.

Metric	Value / Result
Initial UI page load time	< 100 ms
REST API average response time	< 50 ms
Database switch failover latency	< 5.0 seconds (Atlas timeout limit)
Local file CRUD operation time	< 5 ms
Average CPU utilization (idle)	0.1%
Authentication token verification time	< 2 ms

Table 4.1: LMS performance parameters

Test ID	Description	Expected Result	Status
TC01	User registers with missing email or password	400 Bad Request with validation message	Pass
TC02	User logs in with correct credentials	JWT token generated and user returned	Pass
TC03	Access route without JWT in authorization header	401 Unauthorized	Pass
TC04	Student attempts to create a course	403 Forbidden	Pass
TC05	MongoDB Atlas connection timeout	Server falls back to 'db.json'	Pass
TC06	Fetch all courses from course router	List of courses returned	Pass
TC07	Student updates course progress percentage	Progress records upserted without duplication	Pass
TC08	Input standard YouTube watch URL to course manager	URL converted to embed code	Pass
TC09	Instructor edits their own course details	Course record updated	Pass
TC10	Instructor deletes another instructor's course	403 Forbidden	Pass

Table 4.2: LMS software testing summary

4.4 API Design

The REST API endpoints configured on port 5000 are summarized in Table 4.3.

Endpoint	Method	Description
/api/auth/register	POST	Registers a new user with name, email, password and role.
/api/auth/login	POST	Authenticates the user and returns a JWT token.
/api/courses	GET	Returns the complete list of available courses.
/api/courses	POST	Creates a new course (Instructor only).
/api/courses/:id	GET	Returns details of a particular course.
/api/courses/:id	PUT	Updates an existing course (Owner instructor only).
/api/courses/:id	DELETE	Deletes an existing course (Owner instructor only).
/api/courses/progress/all	GET	Returns all progress records of the authenticated student.
/api/courses/:id/progress	POST	Updates course progress percentage and completion status.

Table 4.3: REST API endpoint specifications

4.5 System User Interfaces

The interface screenshots and layouts are represented by the following mockup figures.

The portal login page (Figure 4.1) provides fields for credentials and role selection.

The Student Dashboard (Figure 4.2) display courses and progress bars.

The Course Player page (Figure 4.3) renders lectures and completion toggles.

The Instructor Panel (Figure 4.4) permits course management.

The Certificate page (Figure 4.5) renders printable certificates of completion.

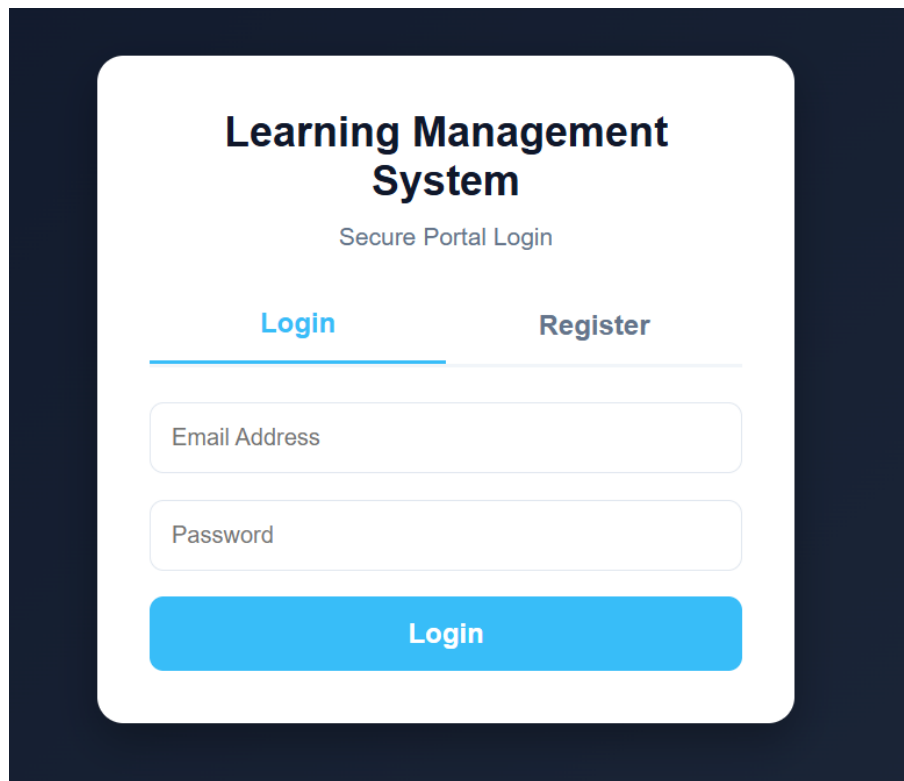


Figure 4.1: LMS portal login page with role-based options.

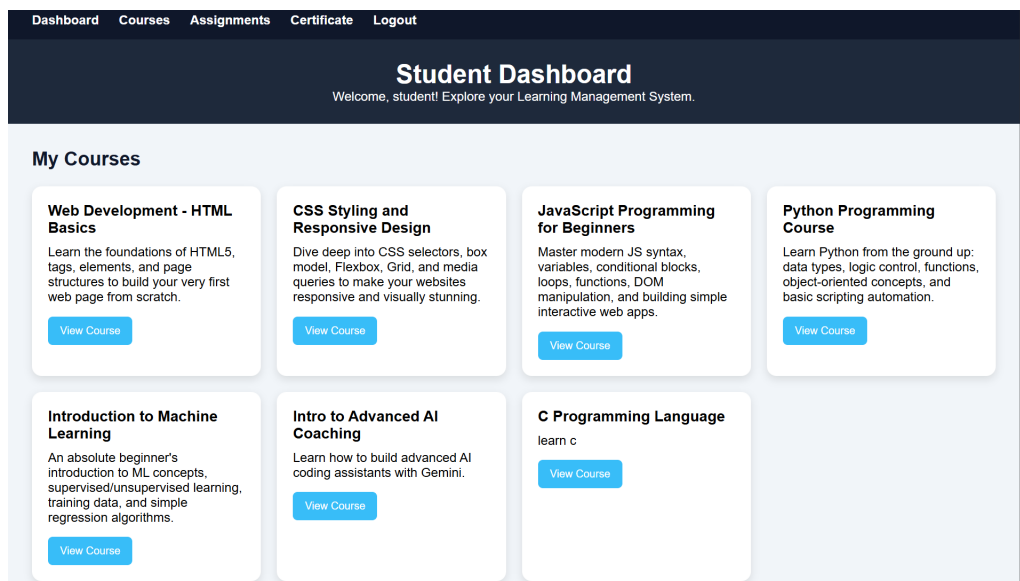


Figure 4.2: Student Dashboard displaying enrolled courses and progress bars.

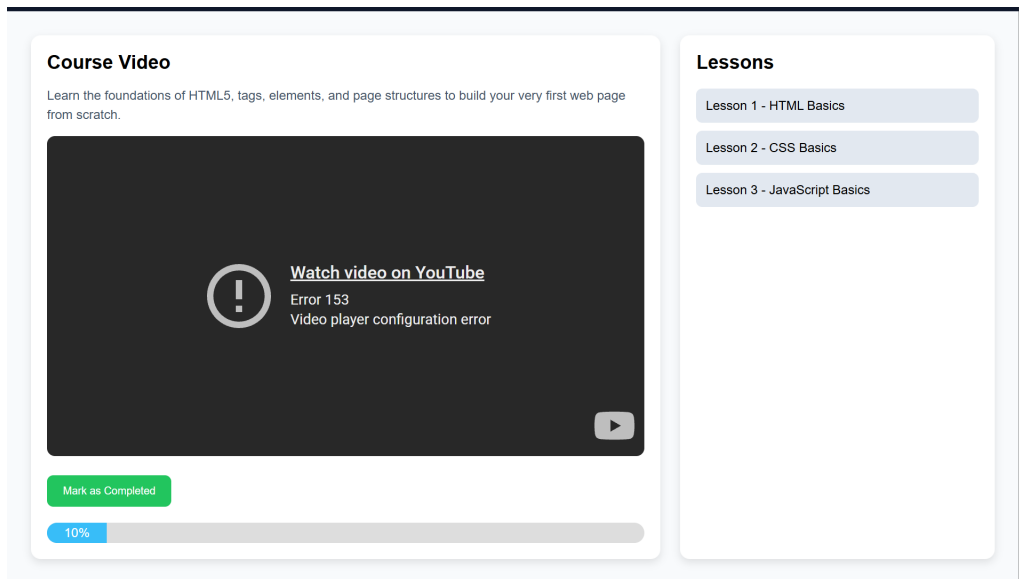


Figure 4.3: Interactive Course Player with embedded YouTube video and progress triggers.

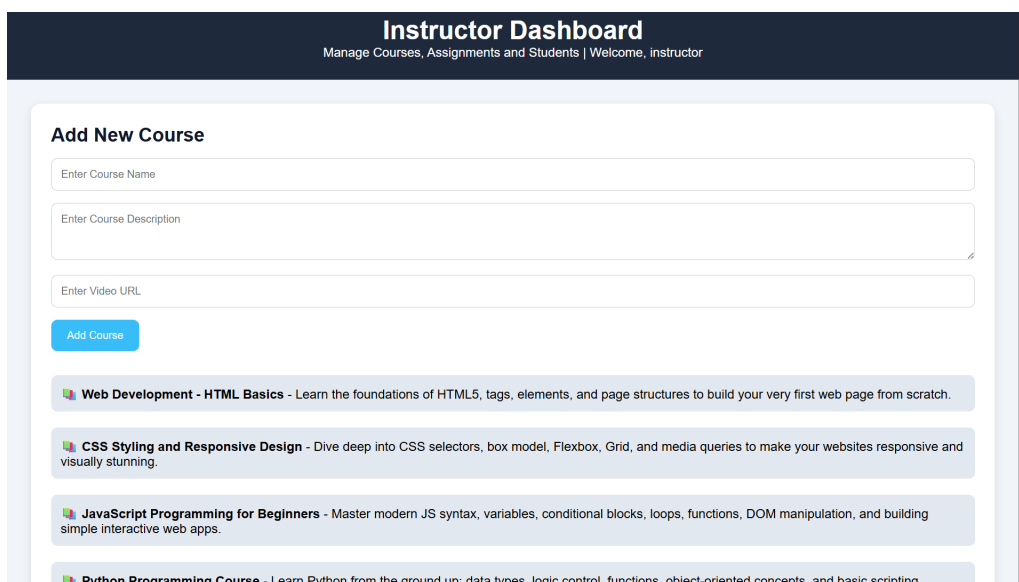


Figure 4.4: Instructor Dashboard interface for course creation and CRUD operations.



Figure 4.5: Renders printable Certificate of Completion.

Chapter 5

Conclusions and Future Work

5.1 Conclusion

In this project, I successfully developed a responsive, full-stack Learning Management System (LMS) that connects students and instructors through clean, functional dashboards. The system provides secure JWT-based authentication, catalog search, video streaming, progress tracking, and certificate generation.

The primary contribution is a custom-designed database fallback adapter (`mockDB.js`) that maintains server operational status during cloud database downtime. If connection to MongoDB Atlas fails, the system automatically redirects query actions to local JSON storage, preserving CRUD operations, schema relations, and generating valid 24-character hexadecimal ObjectIds. This design ensures staging availability and simplifies local demonstration workflows.

The project demonstrates software resilience engineering in web development. By implementing clean, defensive middleware validation, the platform prevents server-side failures and secures user operations based on roles. The interface runs on a responsive glassmorphic theme constructed with plain HTML5, CSS3, and JavaScript, showing that visually appealing designs can be built without heavy framework dependencies.

5.2 Key Contributions

The project delivers the following key contributions:

1. A resilient database routing system that swaps from MongoDB Atlas to a local JSON database fallback (`mockDB.js`) during Atlas downtime.
2. A regex-based YouTube URL converter that transforms standard watch URLs into iframe-embed links.
3. A database-backed course progress tracking schema using a compound index on student and course fields to prevent duplicate logs.
4. A role-based API protection layer using token signature verification and role checks.
5. Responsive interfaces constructed using plain HTML, CSS, and JS.

5.3 Limitations

The system has the following limitations:

- The video player is limited to YouTube streams and does not support direct video file uploads or external storage services like AWS S3.
- The application manages a single primary lecture video link per course and does not support complex multi-module lesson trees.
- The local database fallback utilizes simple file writes and is suitable for staging and testing rather than highly concurrent production scaling.

5.4 Future Work

Future developments will focus on:

1. **AWS S3 Video Upload:** Adding direct MP4 video upload capabilities stored in Amazon S3 buckets.
2. **Multi-Module Course Trees:** Structuring course schemas to support multiple modules, chapters, and quizzes per course.
3. **Vector Database Integration:** Implementing search queries using vector engines for more advanced course recommendations.
4. **FAISS / Milvus Integration:** Replacing the in-memory array filtering with vector databases.
5. **Real-Time Notifications:** Integrating Socket.io for immediate announcements and student-instructor messages.

Bibliography

- [1] M. Cantelon, M. Harter, T. J. Holowaychuk, and N. Rajlich (2014). Node.js in Action. *Manning Publications*.
- [2] A. Holovaty and J. Kaplan-Moss (2009). The Definitive Guide to Django: Web Development Done Right. *Apress*.
- [3] S. Pasquali (2013). Mastering Node.js. *Packt Publishing Ltd*.
- [4] R. Fielding (2000). Architectural Styles and the Design of Network-based Software Architectures. *PhD Dissertation, University of California, Irvine*.
- [5] K. Chodorow (2013). MongoDB: The Definitive Guide. *O'Reilly Media, Inc..*
- [6] D. Crockford (2008). JavaScript: The Good Parts. *O'Reilly Media, Inc..*
- [7] S. Tilkov and S. Vinoski (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6): 80–83.

Chapter 6

Appendices

6.1 API Request and Response Samples

6.1.1 User Registration

Listing 6.1: POST /api/auth/register – Request Payload

```
{
  "name": "Badam Venkatesh",
  "email": "student@example.com",
  "password": "password123",
  "role": "student"
}
```

Listing 6.2: POST /api/auth/register – Response Body (201)

```
{
  "message": "User Registered Successfully",
  "user": {
    "_id": "64b0f1a9b2c3d4e5f6a7b8c9",
    "name": "Badam Venkatesh",
    "email": "student@example.com",
    "role": "student",
    "createdAt": "2026-07-23T01:30:00.000Z",
    "updatedAt": "2026-07-23T01:30:00.000Z"
  }
}
```

6.1.2 Course Progress Updates

Listing 6.3: POST /api/courses/:id/progress – Request Payload

```
{
  "progressPercentage": 100,
  "isCompleted": true
}
```

Listing 6.4: POST /api/courses/:id/progress – Response Body (200)


```
{
  "message": "Progress updated successfully",
  "progress": {
    "_id": "64b0f2c9b2c3d4e5f6a7b8d0",
    "student": "64b0f1a9b2c3d4e5f6a7b8c9",
    "course": "99999999999999999999999901",
    "progressPercentage": 100,
    "isCompleted": true,
    "createdAt": "2026-07-23T01:32:00.000Z",
    "updatedAt": "2026-07-23T01:35:00.000Z"
  }
}
```

6.2 Deployment Instructions

6.2.1 Prerequisites

- Node.js (version 18 or later) and npm installed.
- MongoDB Atlas Account (optional; fallback database requires no configuration).

6.2.2 Backend Setup

Listing 6.5: Commands to build and deploy backend server

```
# Navigate to the backend directory
cd backend/

# Install dependencies
npm install

# Setup environment configuration (.env)
# Specify PORT=5000, JWT_SECRET, and MONGO_URI

# Start developer server
npm run dev
```

The server attempts to connect to the configured MongoDB connection. If Atlas connection parameters are invalid or connectivity times out, the server prints a warning message and falls back to loading database operations from ‘db.json’ locally.

6.2.3 Frontend Setup

Listing 6.6: Commands to run frontend client

```
# Navigate to the frontend directory
cd ../frontend/

# Since the frontend uses static vanilla files, open index.html
  directly in a browser, or run a lightweight local static file
  server:
npx serve .
```

The client registers requests on ‘http://localhost:5000/api’. The login page is served at ‘http://localhost:3000’ or via direct double-click on ‘index.html’.

6.2.4 Accessing the Services

The default locations are listed in Table 6.1.

Component	Default Location
Frontend Client	http://localhost:3000 or local file path
Backend API root	http://localhost:5000
Authentication API	http://localhost:5000/api/auth
Courses REST API	http://localhost:5000/api/courses
Local Database File	backend/db.json

Table 6.1: System access locations